23CSE455 | DESIGN PATTERNS

Design Pattern Identification and Evolution

A case study of a Next.js-based healthcare appointment system

TEAM B2 ▪ CSE D ▪ 11 OCTOBER 2026

Baseline audit ⟶ Pattern-driven evolution ⟶ Cross-language evaluation



Prepared from the repository, source code, test evidence and B2_Report.tex

Presentation roadmap & team

3	3	28	4	100%
PROJECT PARTS	GOF PATTERNS ADDED	CROSS-LANGUAGE BUILDS	LANGUAGES	PART 2 TESTS PASSED

01 Baseline discovery

What was really present in the original code—and what AI incorrectly identified.

02 Software evolution

Adapter for video, Strategy for doctor availability, Observer for reminders.

03 Cross-language comparison

Seven patterns implemented in Java, Python, JavaScript and C++.

04 Evidence & conclusions

Test results, code-size/runtime observations, limitations and lessons.

Team of 5: Gali Rahul ▪ T. S. J. Sriharsha ▪ Kartheek Mudi ▪ P. Jaswanth ▪ Sai Pranay

Case study: the system and the question

Application

CarePulse Healthcare Doctor Appointment Management System
Next.js 14 • React 18 • TypeScript
Appwrite BaaS • Twilio SMS

Baseline architecture

UI forms ⟶ Next.js Server Actions ⟶ Appwrite data operations ⟶ inline SMS side effects

Research question

Are classical GoF patterns genuinely implemented—or are we mistaking framework idioms and conditional code for patterns?

Study method

- 1. Inspect: Review baseline modules and use LLMs to propose pattern candidates.
- 2. Verify: Cross-check candidates manually using AST/control-flow evidence and GoF criteria.
- 3. Evolve: Add three clinical features with formal Adapter, Strategy and Observer structures.
- 4. Compare: Implement seven patterns in four languages; test and measure LOC, complexity and runtime.

Baseline commit: f846cd4c476d48a04fcaeb530b48a77c5295bb92

Part 1 — baseline audit: patterns vs. look-alikes

0	2	5	8
CONFIRMED GoF PATTERNS	PATTERN-LIKE IDIOMS	REJECTED CANDIDATES	UNIQUE PARTICIPANT SYMBOLS

Pattern-like #1 — Singleton

Appwrite Client / Databases / Users / Messaging / Storage are module-scoped and reused. Idiomatic reuse, but no explicit GoF Singleton class.

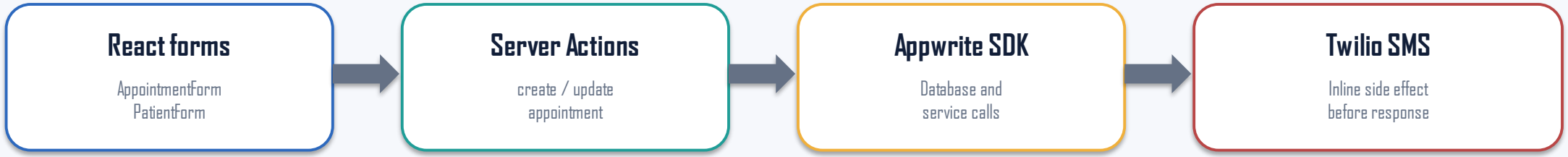
Pattern-like #2 — Factory function

getAppointmentSchema(type) selects a Zod schema. A useful parameterized factory, but not the polymorphic GoF Factory Method.

Five false positives / refactoring opportunities

- Observer: direct SMS call, no subscriber registration.
- Strategy: switch statement inside AppointmentForm, no strategy objects.
- State: raw status union; no state-specific transition objects.
- Command: standalone server-action functions, no command objects or invoker.
- Adapter: business logic calls Appwrite SDK directly.

Baseline architecture: where coupling lived



Problem 1: provider coupling

Appwrite SDK calls sit close to application actions; changing providers affects core code.

Problem 2: conditional complexity

Workflow and availability variation is handled with branching rather than interchangeable policies.

Problem 3: synchronous side effects

SMS dispatch is directly invoked by appointment logic; no event subscribers or durable queue.

Design opportunity: introduce abstractions only where variation, external integrations or side effects justify them.

Part 2 — evolution: three clinical features, three patterns

BEFORE | BASELINE

Procedural & coupled

Direct SDK calls ▪ inline SMS ▪ switch-based policies

AFTER | EVOLVED

Explicit roles & contracts

Target interfaces ▪ replaceable strategies ▪ event subscribers

CH01 — Adapter

Teleconsultation rooms and tokens
4/4 tests passed

CH02 — Strategy

Doctor availability policies
6/6 tests passed

CH03 — Observer

Automated reminders and cancellation
7/7 tests passed

17/17
AUTOMATED TESTS

0
REPORTED REGRESSIONS

+13
CONCRETE CLASSES / UNITS

Pattern 1 — Adapter: isolate video provider details

Client / domain

Appointment action needs standard room details and participant access.

Target interface

IVideoConsultationService defines the stable contract the application expects.

Adaptee / provider

LocalDemonstrationVideoProvider exposes provider-specific data and roles.

The adapter translates between the two contracts

- Data conversion: ISO-8601 timestamp ⟶ epoch milliseconds; duration in minutes ⟶ seconds.
- Role mapping: doctor ⟶ clinician; patient ⟶ client.
- Token normalization: provider-specific session credentials ⟶ domain-compliant VideoRoomDetails.
- Outcome: core appointment actions do not depend on a vendor-specific API.

Evidence: 4 automated tests — room creation, role translation (2 cases) and factory resolution — all passed.

Pattern 2 — Strategy: swap doctor availability rules

DoctorAvailabilityContext

Holds an IAvailabilityStrategy and delegates validation.

StandardBusinessHoursStrategy

Weekday outpatient slots; business hours and lunch blackout.

ShiftBasedAvailabilityStrategy

Doctor-specific shift windows; reject out-of-shift slots.

EmergencyOnCallAvailabilityStrategy

24/7 emergency availability; 15-minute recovery buffer.

Why Strategy fits

- Encapsulation: each rule set lives in a separate implementation of the same contract.
- Runtime flexibility: the context can resolve a policy from doctor availability configuration.
- Open/Closed Principle: add a new policy without expanding a single large conditional.

Evidence: 6/6 tests passed. Limitation: in-memory validation alone cannot prevent concurrent double-booking.

Pattern 3 — Observer: publish appointment domain events

Appointment action

Publishes SCHEDULED or CANCELLED event.

DomainEventBus

Maintains subscribers and dispatches events.

Observers

Reminder scheduler
Cancellation
Audit log

Reminder store

JSON persistence
Idempotency keys

Reminder lifecycle

SCHEDULED

Create T−24h and T−2h reminders

PENDING

Wait until reminder window is due

PROCESSING

Dispatch with retry handling

SENT

Mark completed; idempotency prevents duplicates

- Reliability controls: deterministic idempotency keys, atomic JSON writes, up to 3 retries, cancellation revocation.
- Evidence: 7/7 tests passed across event publication, subscriber dispatch, windows, audit, deduplication, retries and cancellation.

Part 2 deep dive — verification and limits

4/4	6/6	7/7	17/17
Adapter tests	Strategy tests	Observer tests	Total tests passed

Runtime / quality evidence from Part 2 verification report

- Application returned HTTP 200 and loaded the CarePulse homepage.
- Browser verification covered baseline pages, patient registration, appointment booking and admin dashboard.
- Video token endpoint validated accepted roles and rejected invalid roles.
- TypeScript compilation: 0 errors; ESLint: no warnings or errors.
- Production build completed with 11/11 pages compiled.

Demonstration-mode disclosure

Appwrite cloud credentials returned 401; local JSON demonstration storage was used. This is not a live production backend.

Video integration disclosure

The local provider mints demonstration JWT tokens and URLs; it does not provide real WebRTC media streaming.

Deployment follow-up

Production needs a durable distributed event queue and an external scheduled trigger.

What changed after refactoring?

CONFIRMED CLASSICAL GoF PATTERNS



Pattern-like idioms

2 preserved / extended

Mapped collaborations

7 total; roles made explicit

+13	+18	17/17	0
CONCRETE CLASSES / UNITS	UNIQUE SYMBOLS	TESTS PASSED	REPORTED REGRESSIONS

Interpretation

- The design became more explicit and testable, at the cost of additional classes and indirection.
- Formal abstractions were added at high-variation boundaries: provider integration, scheduling rules and notifications.
- The patterns are not automatically "better" everywhere; their value depends on actual variation and coupling.

Part 3 — cross-language experiment design

7	4	28	5 RUNS
PATTERNS	LANGUAGES	IMPLEMENTATIONS	MEDIAN TIMING

Patterns evaluated

Creational

Singleton • Factory Method

Structural

Adapter

Behavioral

Strategy • Observer • State • Command

Language toolchains

Java 21
Explicit interfaces / OOP

Python 3.11
ABC, dataclasses, callbacks

JavaScript / Node 24
ES6 classes, promises, duck typing

C++14
Virtual methods, smart pointers

Cross-language results: concise code vs. explicit structure

Representative total LOC ranges across the seven pattern implementations



Python and JavaScript generally used fewer lines than Java/C++.

Report finding: approximately 38–55% fewer LOC for dynamic-language implementations in the compared patterns.

Key trade-off

Dynamic languages reduce boilerplate; runtime validation becomes more important.

Interpretation caution

LOC depends on idiom and measurement boundaries; less code does not mean better design by itself.

Part 3 deep dive — test coverage and measurement method

Pattern test totals (same behavioral specification across all four languages)

Pattern	Tests per language	Across 4 languages
Strategy	6 tests per language	24 tests total
Adapter	6 tests per language	24 tests total
Observer	6 tests per language	24 tests total
Singleton	2 tests per language	8 tests total
Factory Method	5 tests per language	20 tests total
State	5 tests per language	20 tests total
Command	7 tests per language	28 tests total

Static analysis

LOC excludes blank lines and tests; Lizard reports maximum function cyclomatic complexity.

Timing protocol

One warm-up run, then five process executions; report the median. Process startup affects results.

All 28 implementations passed their pattern-specific test suites in the report.

Benchmark interpretation: timing is not a universal ranking

Python

Often quick in these process-level micro-benchmarks; compact implementations and dynamic dispatch.

JavaScript / Node.js

Fast startup/execution in several measured cases; async promises and built-in collections support event-driven patterns.

Java & C++

Java results include JVM startup/class-loading. C++ results include WSL process invocation overhead.

How to read the numbers responsibly

- Execution time is the median of five process runs after one warm-up, not a production throughput benchmark.
- Different runtimes and startup costs make direct language-level conclusions limited.
- Cyclomatic complexity is maximum function CC; a lower value is useful context, not a complete maintainability score.
- All recorded implementations passed their pattern-specific tests in the report.

Use the measurements to compare these implementations under this setup—not to declare one language universally fastest.

AI-assisted analysis: why human verification mattered

False positive 1

A switch statement was labelled Strategy. Rejected: no interchangeable strategy objects or shared execution contract.

False positive 2

Module-level Appwrite client reuse was labelled formal Singleton. Reclassified as a language-level idiom.

Human correction

Generated Observer skeletons lacked idempotency. Team added deterministic SHA-256 keys and atomic persistence.

Verification workflow

LLM proposes

Candidate patterns

→

AST / code audit

Check roles, interfaces, dispatch

→

Tests

Validate behavior and regressions

→

Human judgment

Accept, reject or qualify

Lesson: a pattern is defined by its collaboration and intent—not by a class name, a switch statement, or a superficial resemblance.

Limitations and production-readiness gaps

Availability race condition

Validation runs in application memory. Concurrent requests may both pass unless the database enforces slot uniqueness / locking.

Demonstration video adapter

Uses a local demo provider; production needs a real WebRTC signaling flow or vendor SDK integration.

Single-process event bus

In-memory event delivery is not distributed across serverless instances; production may need SQS, RabbitMQ or Redis Pub/Sub.

Next engineering steps

- Enforce a database-level unique constraint on doctor + slot timestamp and handle conflict responses.
- Replace the local video provider with a real provider behind the same adapter contract.
- Move reminder events to a durable distributed broker and add monitoring / dead-letter handling.
- Add integration and concurrency tests under realistic deployment conditions.

Conclusion: patterns should solve demonstrated design pressure

Discover carefully

The baseline had two pattern-like idioms, but zero confirmed classical GoF patterns.

Apply selectively

Adapter, Strategy and Observer improved boundaries where vendor coupling, policy variation and side effects existed.

Measure and qualify

28 implementations passed their reported tests; code size and timing must be interpreted within setup limits.

Good design-pattern analysis is not about finding as many patterns as possible. It is about proving the collaboration, explaining the trade-off, and validating that the design solves a real problem.

Presentation plan — five presenters

Gali Rahul	**Slides 1–4**	Introduction, case-study context and baseline audit.
T. S. J. Sriharsha	**Slides 5–8**	Original architecture, evolution overview, Adapter and Strategy.
Kartheek Mudi	**Slides 9–12**	Observer, verification, refactoring outcomes and experiment design.
P. Jaswanth	**Slides 13–16**	Cross-language results, testing, benchmarks and AI-assisted analysis.
Sai Pranay	**Slides 17–20**	Integration evidence, production limitations, conclusion and Q&A.

20 slides total • 4 slides per presenter • all five team members included.

References & Q&A

Primary project evidence

- B2_Report.tex / B2_Report.pdf — master case-study report.
- Part1/ — baseline audit, prompts and pattern inventory.
- Part2/ — evolution changes, UML diagrams, runtime verification and test evidence.
- Part3/ — seven patterns across four languages and benchmark datasets.
- data/cross_language.csv, data/patterns.csv, data/pattern_counts.csv.

Foundational references

- Gamma, Helm, Johnson & Vlissides. Design Patterns: Elements of Reusable Object-Oriented Software (1994).
- Martin Fowler. Refactoring: Improving the Design of Existing Code, 2nd ed. (2018).
- Next.js documentation — Server Actions and Mutations.
- Appwrite documentation — Databases and Node.js Server SDK.
- Lizard — Cyclomatic Complexity Analyzer.

Questions?